Hito 1: Scaffolding y repositorio base

> **Nota: scaffolding vs. boilerplate**
>
> - **Scaffolding**: la estructura inicial y funcional de un proyecto, generada automáticamente con la herramienta oficial de un framework (p. ej. `laravel new`, `ng new`, `npm create vite@latest`). Es justo lo que se pide en este hito.
> - **Boilerplate**: código repetitivo y reutilizable que se copia tal cual entre proyectos (cabeceras de licencia, ficheros de configuración estándar, plantillas de componente…), sin necesidad de ninguna herramienta generadora.

1. Objetivo

A partir de las tecnologías seleccionadas en el **Hito 0**, cada equipo debe crear la estructura inicial del proyecto con el *scaffolding* (o *starter kit*) de cada framework o tecnología elegida. Por ejemplo, si el backend es Laravel, el repositorio debe contener una carpeta `backend/` con la instalación base de Laravel.

El resultado debe ser un proyecto sin lógica de negocio **funcionando en local y documentado**, que cualquier persona ajena al equipo pueda levantar siguiendo únicamente el README. Este hito es la base sobre la que se hará el despliegue en hitos posteriores.

> Antes de generar el scaffolding, configura tu entorno siguiendo la guía VSCode + Copilot CLI.

2. Entregables

2.1 Repositorio Git con estructura mínima

Repositorio en GitHub o GitLab (accesible para el profesor) con esta estructura:

```
proyecto/
├── backend/          <- scaffolding / starter kit del backend
├── frontend/         <- scaffolding del frontend
├── docs/             <- documentación técnica
├── .gitignore
├── .env.example      <- variables necesarias, SIN secretos
└── README.md
```

2.2 Scaffolding funcionando

- El **backend** arranca y expone un endpoint de prueba (por ejemplo `/api/health`).
- El **frontend** arranca y muestra la respuesta de ese endpoint, demostrando que ambas piezas se comunican.
- No se suben al repositorio las dependencias (`node_modules`, `vendor`...) ni ningún secreto.

2.3 README.md

El README es un entregable obligatorio y debe contener los siguientes apartados:

- **Descripción del proyecto** y stack elegido (con el motivo, enlazando al Hito 0).
- **Instrucciones para levantar el proyecto en local**, paso a paso y en orden (ver detalle abajo).

Instrucciones para levantar el proyecto en local

1. Clonar el repositorio.
2. Configurar las variables de entorno: `cp .env.example .env` y qué debe rellenar cada una.
3. Instalar las dependencias de `backend/` y `frontend/`.
4. Preparar la base de datos: crearla, ejecutar migraciones y seeders si los hay.
5. Arrancar el backend: comando y URL/puerto donde queda disponible.
6. Arrancar el frontend: comando y URL/puerto donde queda disponible.
7. **Cómo comprobar que funciona** (por ejemplo, abrir `/api/health` y ver la respuesta en el frontend).

> **Criterio de validación:** las instrucciones deben ser reproducibles por alguien que no conozca el proyecto, copiando y pegando los comandos.

2.4 Documento de instalación y configuración (en `docs/`)

- Pasos seguidos para instalar y configurar el entorno de cada parte del proyecto.
- Capturas de pantalla de las pruebas de funcionamiento (backend, frontend y comunicación entre ambos).
- Problemas encontrados y cómo se resolvieron.

2.5 Historial de Git limpio

- Ramas `main` y `develop` (o la estrategia acordada por el equipo).
- Commits con mensajes descriptivos.
- Contribución visible de **todos** los miembros del equipo.

2.6 Opcional: Docker

Un `docker-compose.yml` que levante backend y frontend (y la base de datos, si ya la tenéis). Si lo incluís, el README debe añadir el comando `docker compose up` y su explicación.

4. Rúbrica (10 puntos)

Aspecto	Peso
Estructura del repositorio y `.gitignore` correcto (sin dependencias ni secretos)	1,5
Instrucciones de arranque en local en el README, completas y ordenadas	1,5
Backend y frontend arrancan siguiendo solo el README	2,5
Comunicación front-back demostrada	1,5
Documentación de instalación en `docs/`	1
Uso de Git (commits, ramas, reparto de trabajo)	2
Total	**10**

5. Revisión cruzada entre equipos

Para comprobar la reproducibilidad, **otro equipo clonará vuestro repositorio y seguirá vuestro README** (en la última sesión del hito). Si no consigue levantar el proyecto, el equipo corrige el README y vuelve a entregar.

6. Temporalización orientativa

Sesión	Trabajo previsto
Semana 1 · Miércoles (2 sesiones)	Creación estructura de carpetas, instalación del scaffolding de backend y frontend.
Semana 1 · Viernes (1 sesion)	Festivo
Semana 2 · Miércoles (2 sesiones)	Conexión front-back, README y documentación en `docs/`.
Semana 2 · Viernes (1 sesion)	Revisión cruzada entre equipos
Semana 3 · Miércoles (2 sesiones)	Resolución de problemas